

Lab Practicum 01: Empirical Parallel Computing

Silicon Profiling, Multi-Core Scaling Limits, and Concurrency Hazards

Duration: 90 Minutes

Format: Solo In-Class Lab

Weight: 5% of Course Grade

STUDENT NAME Orynassar Alinur	STUDENT ID 230103374	HOST CPU / SOC MODEL	OPERATING SYSTEM & KERNEL
----------------------------------	-------------------------	----------------------	---------------------------

Generative AI & Tooling Policy: You are explicitly authorized to use LLMs (ChatGPT, Claude, GitHub Copilot) to generate code scripts in any language (C, C++, Python, Go, Rust, Java). However: All execution timings, memory counters, and hardware cache metrics must be physically benchmarked on your host machine. Fabricated numbers receive zero credit.

TASK 1: Host Silicon Audit & Flynn’s Taxonomy Mapping

15 Minutes | 20 Points

Query your operating system and CPU registers to document physical core topology, cache boundaries, and vector math extensions.

Terminal Audit Commands: Linux: lscpu && lscpu --caches | macOS: sysctl -a | grep machdep.cpu | Windows: Run wmic cpu get /format:list and inspect Task Manager > Performance > CPU.

Hardware Property	Local Measured Value	Hardware Property	Local Measured Value
CPU Model & Architecture	AMD Ryzen 7 6800H	Base & Max Boost Clock	3.20 GHz / 4.70 GHz
Physical Hardware Cores	8	Logical Cores (SMT/Threads)	16
L1 Data / Instruction Cache	512 KB	L2 Cache (Per Core / Total)	4 MB
L3 Shared Last-Level Cache	16 MB	SIMD Vector Extensions	AVX, AVX-2, FMA3

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy:

My host machine can operate in different execution modes depending on the workload.

SISD (Single Instruction, Single Data) is used when a single CPU core executes one instruction stream on one data stream. For example, this can happen during a single-threaded program or a sequential compiler task.

SIMD (Single Instruction, Multiple Data) is used for vector computations, where the same instruction is applied to multiple data elements simultaneously. My AMD Ryzen 7 6800H supports SIMD extensions such as AVX, AVX2, and FMA3, which can improve performance in workloads such as matrix calculations, numerical processing, and multimedia operations.

MIMD (Multiple Instruction, Multiple Data) is used when different CPU cores execute different instruction streams on different data simultaneously. My processor has 8 physical cores and 16 logical threads, allowing multiple programs or parallel processes to run concurrently. The operating system schedules these workloads across the available logical processors.

Therefore, my computer can demonstrate SISD for sequential workloads, SIMD for vectorized mathematical operations, and MIMD for multi-core parallel workloads.

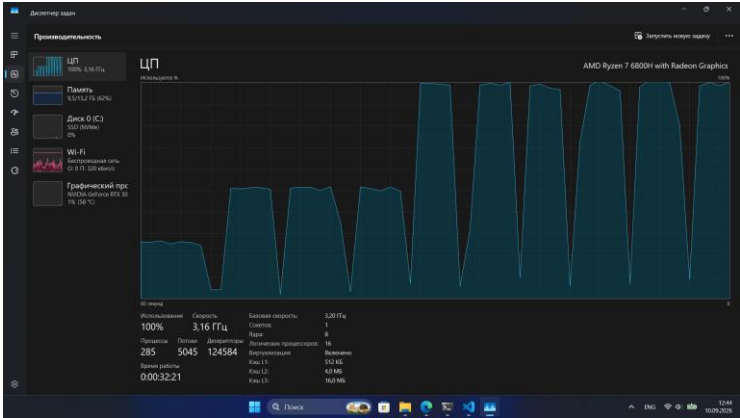
Write or prompt an AI to generate a compute-bound CPU benchmark parameterized by thread count N. Run 3 iterations per thread count and log wall-clock execution time.

Workload Algorithm: [e.g., Prime Sieve to 5,000,000 / Matrix Multiply]				Language & Runtime: [e.g., C OpenMP / Python multiprocessing / Go]		
Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time TN (s)	Speedup SN = T1/TN	Efficiency EN = SN/N

TASK2:Multi-ThreadScalingBenchmark&ContentionWall

35Minutes | 30 Points

N = 1	36.426 s	37.200s	36.759s	36.795s	1.00x	100.0%
N = 2	19.481s	19.800s	21.861s	20.381s	1.80x	90.27%
N = 4	11.602s	11.098s	10.921s	11.207s	3.28x	82.08%
N = 8	6.468s	6.699s	6.655s	6.607s	5.56x	69.61%
N = 16	5.453s	5.574s	5.837s	5.621s	6.54x	40.91%
N = 32	5.796s	5.797s	5.815s	5.803s	6.34x	19.81%



Q2.1: Scaling Saturation & Hardware Contention Analysis:

The benchmark showed strong performance scaling from 1 to 8 threads. The speedup increased from 1.80× at 2 threads to 3.28× at 4 threads and 5.56× at 8 threads. However, the scaling became sub-linear after 8 threads.

The main saturation point appeared at N = 16. My AMD Ryzen 7 6800H has 8 physical cores and 16 logical threads. At 16 threads, the benchmark achieved a speedup of 6.54×, which is only a small improvement compared with 5.56× at 8 threads. This indicates that the workload was already approaching the limits of the available physical CPU resources.

When the thread count increased to **N = 32**, the performance reversed. The average execution time increased from **5.621 s to 5.803 s**, while speedup decreased from **6.54× to 6.34×**. This happened because 32 software processes exceeded the 16 available logical processors. The operating system had to schedule and switch between more processes, creating additional scheduling and context-switching overhead.

The reduced efficiency also demonstrates the contention wall. Efficiency decreased from **69.61% at 8 threads to 40.91% at 16 threads and 19.81% at 32 threads**. Possible causes include CPU scheduling overhead, cache contention and cache thrashing, as well as competition for shared hardware resources. Therefore, the benchmark shows that adding more threads does not always produce proportional performance improvement, especially when the number of threads exceeds the available logical processors.

TASK3:TheUnsynchronizedSharedCounter&RaceCondition Trap

25Minutes | 25 Points

Spawn 10 parallel threads. Each thread executes a loop incrementing a shared global integer 1,000,000 times without atomic operations or locks. Theoretical expected total: $10 \times 1,000,000 = 10,000,000$. Execute 10 consecutive runs.

Run	Measured Output	Error ($10^7 - \text{Act}$)	Run	Measured Output	Error ($10^7 - \text{Act}$)
#1	1.825.402	8.174.598	#6	1.748.774	8.251.578
#2	1.737.800	8.262.200	#7	1.909.463	8.090.537
#3	1.797.916	8.202.084	#8	1.807.960	8.192.040
#4	1.848.300	8.151.700	#9	1.832.573	8.167.427
#5	1.784.744	8.215.226	#10	1.740.172	8.259.828

Q3.1: Read-Modify-Write (RMW) Window

The value corruption occurs because incrementing a shared counter is a Read-Modify-Write operation. The operation consists of three steps: Load → Add → Store. First, a process loads the current counter value from shared memory. Then it adds 1 to the local value. Finally, it stores the new value back to shared memory.

When multiple processes execute these steps at the same time, two or more processes can load the same old value before another process stores its result. For example, two processes may both read the value 100, both calculate 101, and both store 101. One increment is therefore lost.

Because there are no atomic operations or locks, this race condition happens many times during execution. As a result, the measured outputs were much smaller than the expected value of 10,000,000.

Q3.2: The Synchronization Penalty

Unlocked = 470.41 ms vs. Locked = 35,824.20 ms.

The race-free synchronized code is substantially slower because every increment requires acquiring and releasing a lock. Only one process can update the shared counter at a time, while the other processes must wait. This creates synchronization overhead and heavy lock contention.

In this experiment, the unlocked version was much faster but produced incorrect results because of the race condition. The locked version produced the correct result of 10,000,000, but its execution time was significantly higher due to synchronization.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon

15 Minutes | 15 Points

Use your empirical single-thread time (T1) and two-thread time (T2) from Task 2 to derive the parallel fraction p of your workload, then evaluate theoretical performance bounds.

1. Parallel Fraction Derivation (p) Formula: $p = 2 \times (T1 - T2) / T1$ T1 = 36.795s T2 = 20.381s Calculated p = 0.89(decimal between 0.0 and 1.0)	2. Amdahl Theoretical Speedup Ceiling Formula: $S_{max} = 1 / (1 - p)$ Sequential fraction $(1 - p) = 0.11$ Max Theoretical Speedup ($N \rightarrow \infty$) = 9.28×
3. Amdahl 64-Core Projection Formula: $S_{64} = 1 / [(1 - p) + (p / 64)]$ Projected Speedup on 64 Cores = 8.21×	4. Gustafson Scaled Speedup (Weak Scaling) Formula: $SGustafson = (1 - p) + (p \times 64)$ Projected Scaled Speedup (64× Problem) = 57.21×

Q4.1: Strong vs. Weak Scaling Analysis:

Amdahl's Law (Strong Scaling): Models a **fixed problem size**. As core count increases, the time spent in the parallel portion shrinks toward zero, while the sequential fraction $(1-p)$ remains fixed in absolute time. This creates a hard theoretical upper bound ($S_{\{max\}} = 1 / (1-p)$), capping speedup at ~9.28× regardless of how many cores are added.

Gustafson's Law (Weak Scaling): Models a **scaling problem size**, assuming parallel work grows proportionally with available processors ($64\times$ larger problem). The sequential fraction remains constant relative to single-thread execution rather than total execution time, allowing parallel execution to dominate and yielding near-linear speedup (~57.21×).